

From Hardware-in-the-Loop to Hybrid Process Simulation: An Ontology for the Implementation Phase of a Manufacturing System

William S. Harrison, Dawn M. Tilbury, *Fellow, IEEE*, and Chengyin Yuan

Abstract—Hardware-in-the-loop (HIL) is a widely used testing approach for embedded systems, where real components and/or controllers are tested in closed-loop with a simulation model. In this paper, we generalize HIL by combining multiple simulations and real components into a Hybrid Process Simulation (HPS). An HPS is a test setup that contains at least one simulated and one actual component, but may contain many of both. It is implemented such that each simulated component can be swapped out with its real counterpart without making changes to the existing system, and *vice versa*. In this paper, an ontology which provides a conceptual architecture is developed for an HPS, such that a general interpretation of a manufacturing system's implementation is made possible. A formalized application method is then devised for replacing simulations with real processes and *vice versa*. A conceptual architecture is put forth that separates the effect of a component from its spatial essence (volume or mass). This separation allows workpieces in a manufacturing process, for example, to go from the physical world into the virtual world (computer simulation) and back again repeatedly. The conceptual architecture is applied to a small manufacturing line in the following scenarios: replacing a real robot with a simulated robot, replacing a manufacturing cell with a simulated manufacturing cell, and adding a new simulated manufacturing cell to the existing system. These applications successfully demonstrate how an HPS can be used to test a manufacturing system setup with multiple regions of real and simulated components.

Note to Practitioner—Traditionally, in the manufacturing system development process, once the design is completed, a simulation model is built for design validation. Then, the actual system is built. The confidence of the simulation is limited by the accuracy of the simulation model, and testing of the actual system can be time-consuming with high safety risks, such as personal injury and damages to equipment. To accommodate product changes and revisions, there is an ongoing continuous improvement process on the plant floor, such as reconfiguration of manufacturing system equipment, and adding hardware and software into existing processes. This may result in loss of production due to a long validation period with possibilities of damaged equipment. With the HPS approach outlined in this paper, however, the simulated and actual compo-

nents can be validated and tested iteratively; this means each component can be tested with more individual focus, without having to wait until the entire process is complete. In the case of modifying an existing system, simulations can be seamlessly integrated with the real process to evaluate different solutions before a particular option is financially committed. Because the actual components are used during the simulation phase, the overall validation accuracy is less dependent on a single simulation model, while the testing scope is significantly expanded.

Index Terms—Hardware-in-the-loop (HIL), manufacturing automation, simulation, spatial essence, virtual reality.

I. INTRODUCTION

TODAY, manufacturing industries are facing intense global competition because of shorter product life cycles, higher quality standards, and cost constraints. To address these challenges, ongoing research is investigating technologies that facilitate product launch with shorter ramp up times and lower costs. Simulation has emerged as a promising validation solution in the last two decades. There are many simulation tools available to support the testing and validation needs of a process planner for a manufacturing system; however, for a complex system, a single processor-based simulation is not powerful enough to simulate everything accurately. With the typical simulation approach, connections between components may be difficult to replicate in a single simulation environment. Furthermore, simulation models of the entire system may be inaccurate or unavailable, although accurate models may still exist for individual components within the process. For these reasons, a distributed simulation approach is desirable. Future scenarios will call for the use of multiple accurate simulation models alongside the parallel development and implementation of the system as a whole.

In an ideal manufacturing launch process, a proposed system design is built up with multiple different simulation models, and the process planner will bring the process online piece by piece. After the plant is online, the planner wants to run the entire process without workpieces, but still wants to have the appropriate workpiece sensors triggered at the relevant times, which is called a dry run. Once this dry-run without workpieces is completed, the manufacturing system can be put into regular production. Existing simulation methodologies, however, do not address such situations.

Simulation enables better decision making in planning and more accurate performance analysis in the early development stages. System and component simulation models, however, do not always exist, and even with a model, a highly complex con-

Manuscript received June 01, 2010; revised October 01, 2010 and April 30, 2011; accepted May 29, 2011. Date of publication August 18, 2011; date of current version December 29, 2011. This paper was recommended for publication by Associate Editor H. Tanner and Editor V. Kumar upon evaluation of the reviewers' comments. This work was supported in part by the National Science Foundation ERC for Reconfigurable Manufacturing Systems under Grant EEC 95-92125.

C. Yuan is with the General Motors, Global Research and Development, Warren, MI 48093 USA (e-mail: chengyin.yuan@gmail.com).

W. S. Harrison and D. M. Tilbury are with the Department of Mechanical Engineering, University of Michigan, Ann Arbor, MI 48109-2125 USA (e-mail: wsh@umich.edu; tilbury@umich.edu).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TASE.2011.2162584

trol system cannot be fully validated until it is tested against the actual system. Integration of simulation with real components can address both of these drawbacks. Integrated simulation with real components for control testing is more accurate, since it includes the same components that will be used in the final process. Testing with real components makes inaccurate and unavailable models less of an issue and reduces process setup time. Reducing process setup time can then reduce the overall cost of deployment.

This paper presents a formalized methodology for applying the Hardware-in-the Loop (HIL) approach to a test setup with multiple real and simulated components. HIL combines simulated and actual components, leveraging the strengths of simulation, while also reducing the drawbacks that come from a lack of model accuracy. The majority of HIL applications reported in the literature are limited to only one region of real and one region of simulation, most commonly the real controller used with a simulated plant model [1]. Manufacturing systems could benefit greatly from HIL-style testing, but with multiple regions of simulation and multiple regions of real—where each region represents a physical zone of the manufacturing plant floor, and can contain one or more machines, controllers, robots, and/or material handling systems.

One of the interesting challenges for applying HIL in manufacturing systems with multiple simulated regions is what to do about a workpiece that must go in and out of simulated regions repeatedly. When a part moves from a simulated region to a real region, it must necessarily still be simulated. However, the machines and controllers in the physical world must receive the same signals that they would if the real part were present—e.g., part presence sensors, proximity switches, etc., must trip appropriately so that the real region behaves correctly.

The contributions of this paper are twofold. The first contribution is an ontology¹ for Hybrid Process Simulation (HPS) that can describe a manufacturing system during its implementation phase, while it exists as a mixture of simulated and real regions. This new ontology allows for a clear and comprehensive definition of which effects of a component must be emulated in order for the hybrid simulation to adequately represent the final process. The second contribution is a formalized methodology for applying the HPS ontology to replace or integrate simulations with real processes and vice versa. The HPS ontology and application methodology are applied to three different scenarios on a small manufacturing line: replacing a robot, replacing a manufacturing cell (two CNCs and one robot), and adding a manufacturing cell (overhead gantry). These scenarios illustrate the proposed technology and describe how it can directly benefit manufacturing system design and implementation.

The rest of this paper is organized as follows. First, background about HIL in Section II includes a summary of previous work on formalizing a multiple simulation approach. Section III contains the main contributions of the paper: the HPS ontology and the HPS application methodology. The ontology defines the existence of entities by their respective essences and effects, and describes how effects can be emulated so that an individual component need not be aware of the difference between simulated and actual components—the communication at the interface can

be exactly the same. The application of HPS is described in Section IV, followed by conclusions and a brief discussion of future work.

II. BACKGROUND

HIL is a mature field which has been around for 40 years, “ever since the aerospace industry became safety-critical” [2]. Many academic and industrial applications have applied HIL for different testing purposes, and although some formal approaches have been proposed, there does not currently exist a widely adopted formalism that can address the challenges with integrating multiple real regions and multiple virtual regions in a manufacturing system test setup with workpieces moving through the different regions.

A. Formalisms for HIL

The existing literature approaches HIL formalism from two perspectives; they are described here as *fidelity* and *interpretation*. HIL fidelity addresses how well the test setup describes the system or process it represents, oftentimes on a component-by-component basis, and taking an input–output or transfer function view of the components. The fidelity approaches that exist in the literature focus primarily on a particular application. HIL interpretation considers the structure of the HIL setup itself, often focusing on communication between different pieces of the system, and usually with less specification on the internal behavior of the components. The main benefit of the interpretation approach is its potential for a broad class of applications, as opposed to the fidelity approaches which have a more focused set of applications. Although the HPS ontology presented in this paper is a formal interpretation, we also summarize formal fidelity, since it comprises the bulk of the work that has been published on formalization of HIL.

1) *Formal Fidelity Approach to HIL*: Basic suggested a method for assessing HIL test setup fidelity using the concept of transparency [2]. A perfectly transparent system is one in which real components do not feel a difference between the real environment and the HIL test setup. Basic also focused on the interface between the simulation and the real components, and the “robustness of prediction.” However, the definitions were not well-developed enough to be applied to a broad class of problems.

In a follow-up work, MacDiarmid *et al.* provided a detailed approach on how to quantify the accuracy of an HIL test setup [3]. The approach was limited to simulations that could be implemented as linear time-invariant differential equations and used an input–output approach to bound the difference in outputs between the actual and simulated system.

Gawthrop *et al.* showed how to connect mechanical components to an HIL test setup in a way that compensates for dynamic effects of the actuators and sensors used to monitor mechanical devices [4]. The authors provided a sound method for including mechanical components in an HIL test setup, but again use a transfer function perspective.

Boer *et al.* discuss using a backbone architecture called First All Modes All Sizes (FAMAS) that was originally designed to connect different simulations in a distributed architecture [5]. Real components are connected to this backbone, creating an HIL test setup. Though this work formalizes the data sharing for

¹A formal representation of a set of concepts

HIL, it is limited to FAMAS and does not provide any insight into the HIL setup itself.

The PABADIS'PROMISE project used an ontology to develop a novel approach to flexible manufacturing system design and execution [6]. The project also integrated several actual and simulated components for testing purposes, but no general approach or methodology for HIL was presented [7].

In the area of pure distributed simulation for manufacturing, the authors of [8]–[13] address fidelity by specifically looking at the interoperability of different software applications. Their goal was to create a platform for simulation interoperability testing; they did not consider an approach for integrating actual and simulated components.

An HIL fidelity approach specific to mechatronic systems was used by [14], [15]. Fidelity approaches for testing actual and simulated components in manufacturing systems were considered by [16]–[18]. All of these approaches, however, require a custom interface for communication between simulated and actual components.

2) *Formal Interpretation Approach to HIL:* Several researchers have developed frameworks or approaches for defining the interconnections between real and virtual pieces in an HIL system; we group these under the formal interpretation approach to HIL. Many of these frameworks focus on the communication protocol, and several require a centralized system-wide communication infrastructure that connects all of the different pieces of the system.

Auinger *et al.* put forth a comprehensive approach to interpretation formalism [19]. They designed an architecture from the characteristics and requirements of what they call Reality-in-the-loop (RIL) and Soft Commissioning. They specifically outline a set of requirements on the simulation model in an HIL test setup. For example, requirement 1 (R1) states that the simulation of the plant (machines, robots, etc.) should be separate from the simulations of the control environment. Requirement 3 (R3) states that a visualization should be separate from the physical model. R3 goes on to say that “the visualization should be attachable to a specific behavioral model (and therefore to its specific level of abstraction).” Their example, however, is a typical HIL application with a single simulated plant and a real controller (PLC).

Some of the existing work could be considered to address both formal interpretation and formal fidelity. For example, [2] could be interpreted as an early attempt at the interpretation approach, however, the focus was on measuring transparency and robustness of prediction. [5] also attempts an interpretation approach but requires the customized FAMAS backbone for communication.

B. Virtual Reality as the Basis for a Conceptual Architecture

HIL as put forth thus far serves as a testing methodology for hardware equipment and control code. Here, we describe the connection between the basic idea of HIL and Virtual Reality (VR). Section III will outline the new ontology.

1) *HIL and VR Key Objectives:* Manufacturing simulation and VR are approaching a congruence in their application. Simulations are now capable of being portrayed in VR [20]. HIL also shares a congruence with VR's key underlying objective,

which is to provide mediated stimuli to the user, such that he/she can respond as if receiving nonmediated stimuli [21]. The mediated stimuli are represented by signals that manifest themselves in an artificial environment, and mimic nonmediated stimuli which originate from the real environment. Similarly, the key underlying objective of HIL is to provide mediated inputs to a controller or component to see how it would respond in a non-mediated setting, i.e., with the actual components installed and functioning.

HIL and VR methodologies have two important common features that come from the similarities in their key objectives. Within VR, it is said that “scientists want to better understand psychological and physiological processes as they occur in non-mediated settings” [22]. HIL is also used by engineers or process planners to better understand a process or system during its normal operation (nonmediated). The other parallel is the interface; the VR interface is most commonly represented by the connection between the user's eyes and ears and the device used to convey the environment. The interface in HIL is between the tested component and the device that creates the simulated scenario. Virtual reality has a well defined interface called the *communication* interface which is paramount in creating a believable experience [21]. The interface in HIL is equally important, it allows the *behavior* to remain consistent between simulated and actual counterparts. When a simulated component is swapped out for its actual counterpart, communication protocols and connections should remain unchanged, and the same signals should be sent between the components in both directions.

2) *Concept of Presence:* Because HIL and VR share some common features, it makes sense to leverage underlying concepts from VR and apply them to HIL type applications. One important underlying concept in VR is the idea of presence. Lombard *et al.* says that presence is at the heart of virtual reality [22], it “refers not to one's surroundings as they exist, but the perception of those surroundings” [21]. Perception in this case can be described as the user's personal interpretation of the environment.

The concept of presence cannot directly be applied to HIL because it is based on personal interpretation. Machines and humans can receive information about their environment through sensors or senses, but human interpretation is not controllable and machine interpretation is. Therefore, presence must be expanded to a concept that includes, but is not limited to, human-type perception/interpretation.

The following section introduces a new conceptual architecture which can utilize the expanded concept of presence (including humans and machines), while also providing a formalized approach to interpreting the environment that is not based on past experience or personal perception.

III. HYBRID PROCESS SIMULATION (HPS)

This paper presents HPS as a generalization of HIL. HPS has a new conceptual architecture with its own ontology and taxonomy. The ontology is not only generic enough to allow different systems to be translated into conceptual pieces, but also specific enough to retain pertinent information about the system. The ontology is derived from a common understanding of HIL, but has a well defined terminology to support it. HPS

is best suited to support a gradual progression in system development, and is applied on the component level of process deployment. The progression could start with simulation alone, then real components could gradually be added to the simulation, until the system is completely implemented in its final functional form:

simulation $\rightarrow$ HIL $\rightarrow$ physical implementation.

The component-based approach of HPS means that a system level perspective is not required, and furthermore, from the system level perspective, the process will appear as if all components are in place provided that all components are at least simulated.

The following sections start by highlighting the need for new terminology based on the drawbacks of HIL. These drawbacks also highlight the main differences between HPS and HIL. The conceptual structure is then put forth, followed by the technical landscape in which HPS would be most useful.

A. Identified Need for HPS Ontology

The literature supports a general understanding of HIL, but within this understanding there are some variations which point to the need for a new ontology. A large portion of the literature considers HIL to only be for controller testing. In this case, HIL is considered to be when “control hardware is interfaced to the computer simulation and receives inputs from the simulation...” [23]. Another definition says that “HIL simulation involves modeling the plant hardware being controlled, and interfacing the model with the intended controller” [24]. All of the above papers and many others are committed to the idea that HIL is generally for testing controllers [3], [25]–[29].

In addition to the control-testing oriented applications of HIL, there is a more general understanding that includes but is not limited to controllers. One such definition is that “HIL is a scheme that incorporates hardware components of primary concern in the numerical simulation environment” [28]. Another definition say that HIL is “a combination of real-time process simulation connected to a subsystem or component hardware” [30]. These definitions are more general, and define HIL as a testing methodology that may consist of many different types of components real or simulated.

In addition to the differences in the definition of HIL, the literature also has confounding classifications of the terms *software* and *hardware*. One of HIL’s original uses was for electronic control units (ECUs) [31]. ECUs tested with HIL have well-defined software and hardware components, where software represents the simulation and hardware represents the physical controller. Software, however, is not always synonymous with simulation, and hardware is not always synonymous with the final physical implementation. For example, an engine connected to a dynamometer could be considered an HIL test setup by some of the above definitions; however, the dynamometer is hardware which is part of the simulation.

B. Simulated and Actual Components: Structuring a Foundation From HIL for HPS

Instead of hardware and software, in this paper, HIL test setups will be defined using the terms *simulated* and *actual* (Definitions 1 and 2).

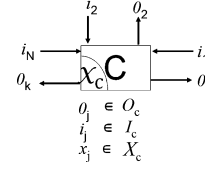


Fig. 1. A component C of a test setup has several interfaces (three in this example); each is a set of inputs and outputs.

Definition 1: Actual: Component or region of a test setup that will be used in the final physical build.

Definition 2: Simulated: Component or region of a test setup that approximates the behavior of its actual counterpart.

The mixed simulated/actual test setup consists of multiple components, where each simulated component has an actual counterpart in the physical implementation, and each actual component is the same as in the final physical implementation. A component is considered to be anything in the test setup that can be simulated, including controllers and machines. Components of the test setup are connected by interfaces (Fig. 1) through which signals (physical or electrical) are transmitted. A component can have multiple interfaces, depending on how many other components are connected to it. An interface is a set of signals (inputs and outputs) where the inputs of one component are the outputs of another. The behavior of a component is defined by the signals that it sends and receives (Definition 3). The intention is that simulated portions of the test setup will eventually be replaced with their actual counterparts.

Definition 3: Behavior: The behavior of a component C of a test setup is a function $o_C = f_C(i_C, x_C)$, where $x_C \in \mathcal{X}_C$ are the states of the component, $o_C \in \mathcal{O}_C$ are the outputs, and $i_C \in \mathcal{I}_C$ are the inputs.

As discussed in Section III-A, software and hardware do not accurately describe the components of an HIL’s progression from simulated components to actual components. An HIL setup often includes only one simulated region and one actual region. For the preceding reasons, the term HPS is used to replace HIL (Definition 4).

Definition 4: Hybrid Process Simulation: A test setup that contains at least one simulated and one actual component.

“Hybrid” refers to the mix of simulated and actual components. “Simulation” refers to test setups that have at least one simulated component.

HPS differs from conventional HIL because there can be multiple simulated components in a test setup. Conventional HIL (with typically one simulated component) can be considered a subset of HPS. Fig. 2 shows how an HPS exists between complete simulation and the actual system during a development cycle. Each simulated component $\tilde{C}$ must have the same inputs and outputs (and approximate behavior) as its actual counterpart C .

C. HPS Ontology

This section presents the ontology of HPSs. Existence will be divided into two domains, the digital and the physical. Entities can exist in one or both domains. The interactions between entities are defined by their effects, which come from their essences.

When a simulation replaces an actual component in an HPS, the

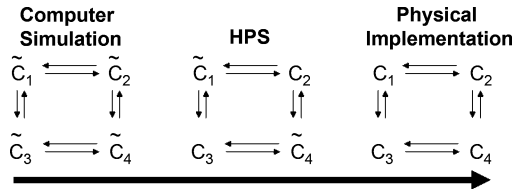


Fig. 2. An HPS consists of multiple components. Some are simulated $\tilde{C}$ and others are a part of the final physical implementation C . The large arrow shows the progression during a system's development from all component simulation, through HPS, to complete physical implementation.

effects of the actual component must be emulated by the simulation, so that the other components in the system do not realize that the actual component is not present.

The most important purpose of an HPS is testing; this may include component capability, control code correctness, or system performance. The essential aspects of testing are providing inputs and monitoring the outputs. For the test setups considered here (manufacturing systems), monitoring the behavior is relatively straightforward; the challenging part becomes controlling the inputs.

The first step in developing this conceptual architecture is to consider what inputs really are. Conceptually speaking, inputs are, in most cases, effects from other regions of the test setup. This fact suggests that *effect* is the most important part of an HPS, and thus should serve as the linchpin of the architecture.

1) *Essence and Effect*: Two types of *effects* are considered here. The first type is from the point of view of the physical domain, which includes anything that can be physically interacted with. Because the effects of physical interaction are echoed in Newton's third law, this type of effect is termed *Newtonian effect* (Definition 5).

Definition 5: Newtonian Effect: The change of physical characteristics.

Examples of Newtonian effects are the changing of location, pressure, shape, and temperature. The second type of *effect* is from the point of view of the digital world. Here the digital world includes any information that resides in computers. Because procedural programming languages focus more on functions and the change of data rather than the data itself, this effect is termed *procedural effect* (Definition 6).

Definition 6: Procedural Effect: The change of electronic/computer information.

Procedural effects include any changes of information stored in computers, such as incrementing a part count.

The *procedural effect* and the *Newtonian effect* can be connected directly through mapping (Fig. 3). An example of this type of mapping would be to use a desktop computer to transfer money. The Newtonian effect of hitting the keys and moving the mouse is directly mapped to the procedural effect, which is the changing of account information.

The existence of an effect suggests a cause. The cause of an effect from the physical point of view involves mass and volume. Mass or volume could then be considered the essence of existence. For this reason *essence* is complementary to effect, and also defines effect (Fig. 3). In the physical world, *spatial essence* (Definition 7) (mass/volume) is complementary to *Newtonian effect*.

Definition 7: Spatial Essence: The physical volume or mass representing an entity in the physical world.

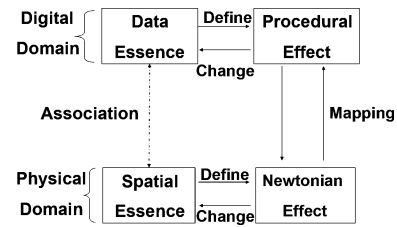


Fig. 3. Domains are comprised of essence and effect. The only direct connection between two different domains is the mapped effects.

In the digital world, *data essence* (Definition 8) is complementary to *procedural effect*. Data essence represents the information itself, and might include the part number in a manufacturing process.

Definition 8: Data Essence: The electronic information representing an entity in the digital world.

Data essence implies procedural effect and spatial essence implies Newtonian effect. An HPS utilizes these implications by considering how a Newtonian effect can be achieved without a spatial essence

$$\begin{aligned} \text{Data Essence} &\implies \text{Procedural Effect} \\ \text{Spatial Essence} &\implies \text{Newtonian Effect.} \end{aligned}$$

Spatial essence and data essence are connected through association only (Fig. 3). A social security number (data essence) may electronically represent a person (spatial essence), but its connection is not intrinsic. It is accepted that the number is connected; however, that connection is strictly an agreed upon association.

Essence and effect are complementary within a *domain*, and entities exist in one or more domains. In this paper, only two domains are considered: the *digital* and *physical*, each of which may have multiple regions.

Definition 9: Physical Domain: The complementary pair of *Newtonian effect* and *spatial essence*.

Definition 10: Digital Domain: The complementary pair of *procedural effect* and *data essence*.

2) *Regions, States and Transitions*: An HPS potentially has many different simulated and actual regions. Actual regions may exist in both domains, and computer simulated regions only exist in the digital domain. In addition to the simulated and actual regions of an HPS, the workpiece itself exists as an actual or simulated entity. Regardless of whether a workpiece starts as being actual or simulated, it should be capable of moving through any region of an HPS (Fig. 4). A workpiece can also exist in one or more domains. We consider the *state* of an entity to be the domains (essence and effect) in which it exists. We also consider that workpieces may transition between regions that occupy different domains.

Workpieces that transition between regions must change their state (essence and effect) to match the domain of their surroundings. An entity can transition in and out of the physical domain numerous times; however, after it transitions into the physical domain, it may not gain spatial essence but it can regain Newtonian effect.

To illustrate the connection between a workpiece's state and its surroundings, consider a manufacturing HPS. Assume that

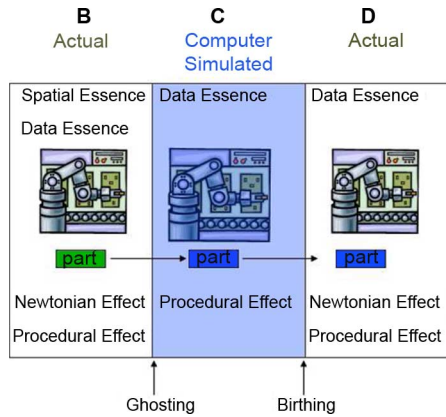


Fig. 4. The essence and effect of a workpiece are largely dependant on whether the region the workpiece occupies is simulated or actual.

the workpiece will interact with various different machines in the process as it moves down a single track serial line. The essence and effect of the workpiece depends on the region it currently occupies. Fig. 4 shows how a process could start with a region containing all actual physical machines, followed by a region with all simulated machines. These regions alternate as the workpiece moves down the line. When the workpiece is in the physical world it may exist in both the physical and the digital domain, and have both Newtonian and procedural effect as well as data and spatial essence, which is termed *dual presence*. When the workpiece is in a computer simulation, however, it will only exist in the digital domain because it cannot have spatial essence in a computer simulation. The transition into the simulated region with a loss of spatial essence is termed *ghosting*. When the workpiece reaches the next region in the test setup where it interacts with components that have physical essence, it still will not have spatial essence but it should regain Newtonian effect. The transition from the simulated region back into the actual region with Newtonian effect is called *birthing*.

Definition 11: Dual Presence: An entity with both spatial and data essence.

Definition 12: Ghosting: The act of losing spatial essence.

Definition 13: Birthing: The act of gaining Newtonian effect.

When a workpiece leaves a computer simulation and goes through birthing into the actual system's physical domain (Fig. 4 regions C to D), the workpiece must be simulated and must still interact with the physical domain. The simulated workpiece can interact with the physical domain because it will have Newtonian effect (Figs. 4 and 5, region D). Because the workpiece maintains Newtonian effect, the other components of the system are unaware of the workpiece's lack of spatial essence. This type of Newtonian effect is accomplished through signal emulation, allowing all entities that interact with the physical workpiece to have the same exact interactions with the simulated workpiece. An example of such an interaction might be a workpiece presence sensor that must sense both the physical workpiece and the simulated workpiece. The simulated workpiece has Newtonian effect in the physical domain, because the simulated workpiece activates the same sensor signal effect a real workpiece would activate.

As a workpiece moves through an HPS it may have many different states. Fig. 5 shows the possible states of a workpiece

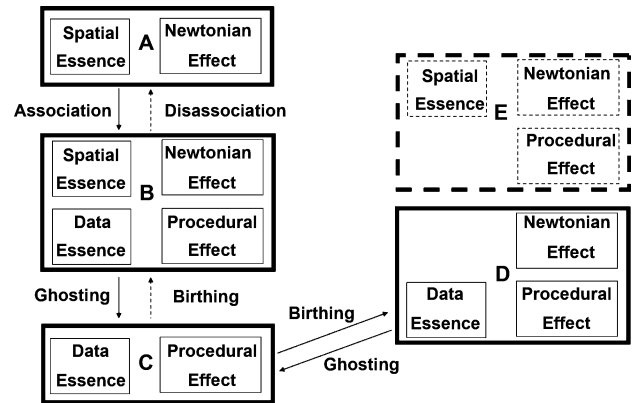


Fig. 5. Components of an HPS can exist in different states with different combinations of essence and effect.

in an HPS. The workpiece may gain or lose essence and/or effect as it transitions from one region (simulated or actual) to another. Dashed lines represent uncommon states or transitions, and the solid arrows represent the most common transitions in an HPS. The solid arrow from regions A to B in Fig. 5 represents an entity that gains essence and effect in the digital domain, which might happen by giving a product in a delivery system a bar code or tracking number; the bar code is then connected to the physical workpiece through association. The solid arrow in Fig. 5 from regions B to C represents ghosting, which is a transition where an entity leaves the physical domain. This transition would happen when an entity goes from an actual physical region of an HPS to a completely computer simulated region. The dashed arrow in Fig. 5 from regions C to B represents a transition where an entity in the digital domain also enters the physical domain. A physical test workpiece would need to be added at the boundary between the two regions and would likely come from something that is not a regular part of the system (e.g., a person in charge of validation). The dashed arrow from regions B to A in Fig. 5 represents the disassociation of an entity's digital and physical domain. An example of this disassociation might be after a workpiece is complete, and no longer needs individual tracking. The workpiece is still physically present and will be delivered, but its individual information holds no value. Region E in Fig. 5 represents an entity that can affect the digital domain through mapping but has no representation in the digital domain. Region E is the only dashed lined state because it is not a useful state for validation purposes, and thus no transitions to or from it are shown.

3) Application: Now that the conceptual architecture has been put forth, the logistics of such an application can be addressed. For this description, consider the simple example illustrated in Fig. 4.

The following are a set of requirements for a component simulation in an HPS similar to Fig. 4, region C.

- The simulation should be real-time.
- The simulation should connect to the actual system's communication infrastructure.
- The simulation interface should be the same as that of the actual component it simulates.

A computer simulation such as this could be housed on a desktop or a laptop computer as long as it has the correct interface card for communication. Ideally, such a component

simulation would come from the vendor of the actual component. Vendors have the most incentive for creating such a component simulation, because the simulation could be used to promote the purchase of their actual components.

4) *HPS Ontology Summary*: The HPS ontology presented in this section allows the existence of an entity—whether simulated or actual—to be defined by its essences and effects. When an entity has spatial essence, its Newtonian effects can be felt by other entities that it interacts with in the physical domain. Similarly, when an entity has data essence, its procedural effects can be felt by other entities that it interacts with in the digital domain. The HPS ontology allows for entities to have dual presence, existing in both the physical and digital domains. Dual presence can be accomplished through association and mapping, such as when a part with spatial essence is linked to a database record with its type, ID, history, etc. In addition, an entity with only data essence may also have Newtonian effects (in the physical domain) through mapping and signal emulation. The HPS ontology can be used to describe the movement of an entity as it transitions through different domains of existence—for example, when a part transitions from a physical region into a simulated region on a manufacturing line, it must lose its spatial essence. When the part transitions back into a physical region, it must regain its Newtonian effect in order to interact appropriately with the other entities in the region. The HPS ontology can be applied to all of the entities in a manufacturing system, including the machines, controllers, and workpieces, to define how they interact with each other, whether they are actual or simulated. Using these definitions, different combinations of actual and simulated can be tested, with the potential to streamline the system validation process. Section IV will describe the application of the HPS ontology to three different scenarios on a small manufacturing testbed.

D. HPS Integration Methodology

This section presents the methodology to apply HPS to the deployment of a manufacturing system. Two different variations of the methodology are described. In one scenario, the system starts as a pure simulation, and the actual entities are added incrementally until the final system is complete. In the other scenario, the system starts as all actual, and a reconfiguration first adds a simulated entity which will then be swapped out for its actual counterpart. Because HPS can be applied to test setups including machines, controllers, and many other types of components, the term entity will be used here generically.

In both scenarios, the simulations must have the capability of being connected to actual components. The HPS approach cannot be applied to a simulation that is completely standalone with no outside connections.

To replace a simulated entity with an actual one, first several steps must be completed.

- 1) Identify the entities involved in the process.
- 2) Define the essence and effect of each entity.
- 3) Define the boundaries that outline the regions of the test setup that will be replaced; regions can contain one or more entities.
- 4) Define the behavior of each entity.

After the above have been defined, the following steps are repeated until the process becomes all actual.

- 1) Choose the simulated entity that will be replaced with the actual entity.
- 2) Acquire the actual entity (hardware, software, and/or components).
- 3) Remove or deactivate the simulated entity where the actual entity will be added.
- 4) Connect the actual entity to the test setup.
- 5) Modify the setup as needed to accommodate both the actual and simulated entities so that the HPS runs properly. For example, the actual entities may need sensor signal emulation so that they function correctly with workpieces that have no spatial essence.
- 6) Run the test setup for validation.
- 7) If more simulated entities remain, go to step 2, or, if validation fails, return to step 5.

Scenario 2 starts from a test setup that has some or all actual entities. A simulated entity will replace part of or be added to the test setup. The simulated entity represents a potential change to the existing test setup; after validation with the new simulated entity, it will be replaced with its actual counterpart. The simulated entity must have the capability of being connected to the actual test setup in the same manner as its actual counterpart.

To replace an actual entity by adding a simulated entity, the following steps are used.

- 1) Identify the entity to be replaced.
- 2) Define the essence and effect of the simulation that will be added.
- 3) Define the behavior of the entity.
- 4) Define the interface (logical and physical).
- 5) Acquire or build the simulation.
- 6) Remove or deactivate the actual entity where the simulation will be placed.
- 7) Connect the simulation to the test setup.
- 8) Modify the setup as needed to accommodate both the actual and simulated entities so that the HPS runs properly. For example, the actual entities may need sensor signal emulation so that they function correctly with workpieces that have no spatial essence. An example of how this is done can be found in Section IV-B.
- 9) Run the test setup for validation.

After the validation is complete, when the actual change is ready to be made, go to scenario 1 step 2. If the validation process cannot be completed successfully, the chosen simulated component may not be a viable option for replacement. Another component simulation with different characteristics can be tested in place of the previous until a viable option is reached. In addition, if the component simulation lacks some functionality that the actual component will contain, the validation with the HPS may not be successful.

One variation of scenario 2 is to add a simulated entity to the existing test setup instead of replacing one. For this variation the first step becomes:

- 1) Identify the entity to be added.

All other steps remain the same with the exception of step 6, which can be skipped.

E. Technical Landscape for an HPS

The HPS accommodates many different simulated components interfaced with many actual components. This type of im-

plementation begs the questions, from where do these simulations originate, what form do they take, and what information do they need. The authors foresee a scenario where the simulations are developed and supplied by the vendors of the actual components they represent. The manufacturers are motivated to provide accurate, reasonably priced simulations because the simulation models will be used to support purchasing decisions. A manufacturing process planner could then bring in multiple simulation models from different vendors and compare them in a “try before you buy” scenario. This approach has several benefits to the manufacturing process planner, one being that it allows the process planner to try many different solutions before financially committing to a particular one. Another benefit is that code used to program the simulation can directly be ported to the actual component. For example, Fanuc’s Roboguide simulation software allows the code from the simulation to directly be uploaded to the actual robot.

There are two drawbacks to the “try before you buy” scenario. First, each different vendor component simulation that serves as a possible component replacement or addition may be very different, requiring some learning curve for its implementation. There are both advantages and disadvantages to finding out about implementation difficulties. It will undoubtedly add to the time needed for the entire implementation; however, information about ease of implementation is useful too. Because the overall investment has not been made yet, a process planner can decide on a different solution if it looks like the difficulty of implementation outweighs the gains from production performance. Second, if there is a large variety of components in the process, it may be too time consuming to try multiple different vendor solutions for each component. The HPS approach can still be employed by acquiring the component simulations, but trying multiple vendor options for the same component should be used sparingly in accordance with time and work capacity constraints.

1) *Simulated Components*: One inevitable implementation barrier is the proprietary nature and form of the simulations from the vendors. The term simulation is used here broadly to include both the software that executes the simulation and the hardware platform that the simulation sits on. For example, a “soft PLC” (programmable logic controller), which is a viable PLC option in its own right, can be considered as a hard PLC simulation with identical function and connectivity. A process planner could use soft PLCs to test and compare competing solutions. All of the comparisons could be done using the same desktop and interface cards, while the program being run is switched from one solution to another.

This type of simulation is not restricted to PLCs. Section IV will discuss how this type of simulation can be done with a robot. The important point is that migration from simulation to final implementation needs no internal alteration. It facilitates system level validation (how the component integrates into the larger system) instead of isolated validation (the component alone).

2) *Actual Components and Signal Emulation*: Future actual components should be designed with the capability of handling workpieces without spatial essence (signal emulation mode). After a workpiece has gone through ghosting for the first time, all signal emulation capabilities downstream of the ghosting

should be active. Vendors could be motivated to give such capabilities because it should make their components more flexible for system development and changes.

The method by which sensor signal emulation is activated can vary. The easiest method is to pass a token/variable with the workpiece that lets the actual components know that there is a workpiece without spatial essence present.

The behavior of the actual component in signal emulation mode will be up to the vendor. In the example covered in Section IV, the entire robot program is run in its final state. The part presence signal in the robot program is emulated in the robot when a simulation token is activated, causing the appropriate sensor response at the appropriate time.

The above approach works well when not all sensors are accessible from one location. If all the sensors reside on a single network, signal emulation can be handled by an external signal emulator program that emulates the procedural effects at the appropriate times.

Other ways of dealing with simulated workpieces have been explored. [32] use a conveyor buffer system to allow rough workpieces to appear when a virtual workpiece goes from a simulated component to an actual component, and then disappear when the opposite happens. This solution is effective, but requires that workpiece emulation be handled on a higher system level rather than on a local component level.

For simple part presence sensors, emulation mode may require few code changes. For more advanced sensing such as robot vision, the component may require an emulation mode runtime environment. This runtime environment may require that complex sensing capabilities that cannot be emulated be bypassed completely.

Signal emulation mode for actual components does not allow for the validation of very specific sensor functionality within the system; however, it does allow for the testing of the system as a whole, which is arguably the more challenging endeavor. Validation of an isolated machine (component validation) can be done effectively without the presence of the system, and will likely be as effective whether done alone or as part of the larger infrastructure.

An HPS allows for large-scale system level validation as well as some validation on the local level. Though an HPS as proposed cannot test sensor performance or precise physical functionality, it does allow for validation on the system control level. This capability also gives the process planner the opportunity to integrate components into a system that could be in a mostly simulated phase, reducing the ramp-up or changeover time.

3) *Limitations*: An HPS can test system connectivity as well as validate control on different levels of the system’s hierarchy. Extending HPS to include robustness and repeatability analysis on a component basis is possible but would require the simulation to have stochastic characteristics or be based on historic statistical data. Internal errors could be introduced in the simulation models by the designers to emulate breakdowns or problems with the actual equipment. How the simulation handles such internal errors would be up to the designer of the simulation model. While simulations with these capabilities could be constructed, this type of information will most likely have to come from data supplied by the vendor of the equipment. Though testing system functionality for process errors such as

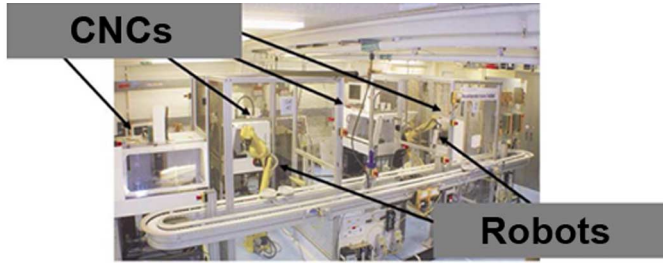


Fig. 6. A photo of the serial-parallel line. Cell 2 is on the left and Cell 1 is on the right.

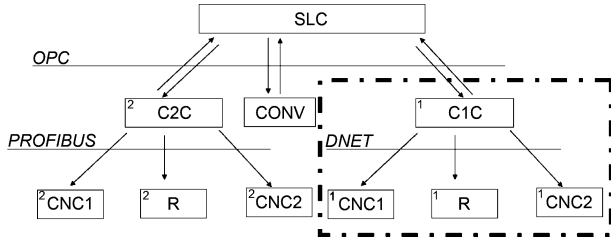


Fig. 7. The control hierarchy of the RFT, where R, CNC, CONV, and C1C stand for robot, CNC machine, conveyor, and Cell 1 controller, respectively. The numbers in the corner of the blocks specify which cell the block is in.

robot gripper faults and other equipment specific errors are important, the focus of this paper is HPS for testing integration and connectivity.

Errors that arise from the real components or the interaction between components in the HPS can still occur. The error should be observable using the same real-time diagnostic software used to monitor the actual system, or by observing that the HPS ceases to run correctly or at all. If the error is not visible for any of the aforementioned reasons, the HPS approach will not be useful in detecting the error. An example of such an error would be a cutting process that is done incorrectly resulting in a misshapen workpiece. If ghosting happens right after the cutting process, an error may not be detected.

The authors realize that simulations which connect and function the same as the processes or components they are meant to simulate do not yet exist from all vendors. It is possible, however, for process planners to create such simulations as described in Section IV. As stated earlier, the vendor of the actual component is best suited for developing these simulations; however, a third party may also be a good candidate for creating such simulations.

IV. IMPLEMENTATION EXAMPLES

A. Reconfigurable Factory Testbed (RFT)

In this section, the HPS ontology and the formalized application method put forth in this paper are applied to the Reconfigurable Factory Testbed (RFT) [33]. The RFT's physical components include two robots, four CNC machines, and one conveyor, which make up the serial-parallel line (Fig. 6). The robots and CNCs are equally divided into two cells (Cell 1 and Cell 2) (Fig. 7), which are coordinated by the System Level Controller (SLC).

The RFT represents a process that is initially a completely *actual* manufacturing system. The RFT becomes an HPS in the

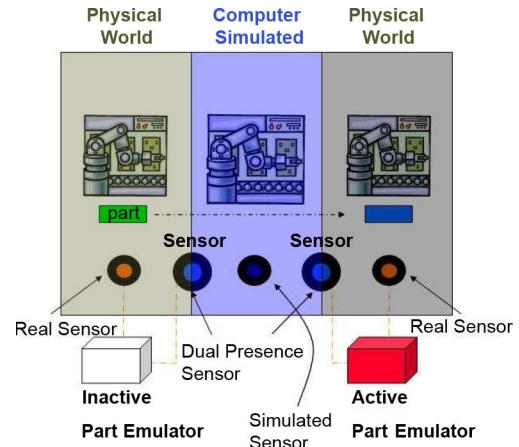


Fig. 8. Workpieces can leave the simulation and enter the physical domain by emulating all Newtonian effects.

testing scenarios presented next. In these scenarios, *simulated* cells and components are integrated into the existing process. The entities (blocks in Fig. 7) in the RFT will be abstracted and modeled as discrete-event systems. It is assumed that each entity can observe the internal state of those below it in the hierarchy.

In this section, the term “workpieces” will refer to the pieces of wax, and the regions and components in the RFT will be identified by name (such as Cell 1 or CNC1). Because Cell 1 and Cell 2 are very similar, the focus of the implementation is on Cell 1. Entities in Cell 1 include the following: wax workpiece (W), Cell 1 controller (C1C), CNC machine 1 (CNC1), CNC machine 2 (CNC2), and robot (R).

The HPS test setups addressed in this paper represent three variations of scenario 2 [starting with a mostly actual system and replacing parts of it with simulated entities (Section III-D)]. Scenario 2 is chosen because it shows the highest contrast between the HPS approach and common practices within manufacturing. Customarily, if modifications for an existing process line were required, process planners would not implement real-time “plug and play” simulations. They would instead buy the equipment and/or software/hardware directly. Buying the actual components directly eliminates the ability to evaluate different solutions using different simulations, and also could result in damage to personnel and equipment due to unexpected errors. Scenario 2 is also chosen because, once finished, it then becomes scenario 1 (replacing a simulated entity with an actual entity), step 6, which is the step the process planner would already have to accomplish. The following subsections will define the behavior, interface, essence, and effect of the entities in Cell 1.

1) *Workpiece*: One challenging aspect of an HPS is dealing with entities that must travel from simulated regions to actual regions (Fig. 8). Workpieces must be processed by different regions of the manufacturing system whether they are actual regions in the physical domain or computer simulated regions in the digital domain. Fig. 8 shows sensors at the boundaries between a computer simulated region and an actual region. These sensors activate workpiece emulation (in the physical domain) when a workpiece crosses the boundary from a computer simulated region to an actual region. The sensors must be dual present (existing both in the simulated and actual

region) because simulated workpieces must trip the sensor when they go from a simulated region to an actual region, and actual workpieces must trip the sensor when they go from an actual region to a simulated region. Dual presence sensors work by triggering the appropriate procedural effect when they are activated, regardless of whether the sensor is simulated or physically present. On the RFT when a physical sensor is activated (Newtonian effect) by a physical workpiece, the robot, for example, is alerted (procedural effect) and begins its pick and place operation. When the robot and the sensor are simulated, the physical (spatial essence) robot is no longer present but the physical sensor remains. When the physical sensor is activated (Newtonian effect) so too is the simulated sensor (procedural effect) alerting the simulated robot to begin its pick and place operation. The link between the physical and simulated sensor is a mapped effect. When the simulated workpiece is placed near the simulated sensor by the simulated robot after processing is complete, the same procedural effect as that caused by the physical sensor will occur. Ghosting and birthing occur at the dual presence sensor locations.

Wax trains are the end product of the RFT. Initially, each wax train starts as two blocks (workpiece 1 and workpiece 2). Each wax block set is processed in series in each cell. If the workpiece has no spatial essence just before it reaches Cell 1 the Newtonian effects must be emulated.

The Newtonian effect of a workpiece in Cell 1 is sensed by an inductive presence sensor located on the robot gripper. The emulated signals come from a positive 24 V applied to the part presence sensor inputs.

2) *System Level Controller 1 (SLC1)*: SLC1 is the coordinator for the entire process. The behavior of SLC1 is a function $o_{SLC1} = f_{SLC1}(i_{SLC1}, x_{C1C}, x_{SLC1})$, where o, x , and i are discrete states and events, as described in Fig. 9. SLC1 (used for HPS testing) only has one interface. The interface input and output signals are communicated through OPC. SLC1 exists in the *digital domain*; its *data essence* is the program code and its *procedural effects* are the output commands given in Fig. 9.

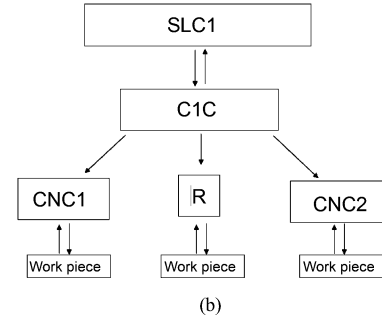
3) *Cell 1 Controller (C1C)*: C1C, which acts as the coordinator for the robot and the CNC machines, will remain in its *actual* form throughout testing. The behavior of C1C is a function $o_{C1C} = f_{C1C}(i_{C1C}, x_{CNC1}, x_R, x_{CNC2}, x_{C1C})$ where o, x , and i are given in Fig. 9. C1C does not have inputs from the CNCs and robot because it can see their internal state. The highest level interface is to SLC1, where the communication is done through OPC. The lower level communications to the CNCs and the robot are over DeviceNet.

C1C exists in the *digital domain*; however, some of its *procedural effects* are mapped to *Newtonian effects* (Fig. 5, region D). This mapping is for the communication between C1C and the CNCs. Mapping is needed because the CNCs receive 24 V signals for control. These signals are different from the normal C1C's DeviceNet communication signals. The voltage signals require a conversion from the digital bit data of the DeviceNet signal to a voltage, which is handled by a DeviceNet I/O block. Similar to SLC1, C1C's *data essence* is the program code and its procedural effects are the output commands (given in Fig. 9).

4) *CNC and Robot*: The CNCs are responsible for cutting the wax blocks, and the robot is responsible for transporting the wax blocks between the conveyor and the CNCs.

C1C	SLC1	R	CNC
States (x_{C1C})	States (x_{SLC1})	States (x_R)	States (x_{CNC})
• Empty • Part 2 Loading • Part 1 Cutting • Part 1 Unloading • Part 2 Cutting • Part 2 Unloading • Part 1 Loading	• Part 1 in Cell 1 • Part 2 in Cell 1 • Both Parts in Cell 1	• Robot Idle • Loading part 1 • Loading part 2 • Unloading part 2 • Unloading part 2	• Part 1 in Process • CNC Idle
Output (o_{C1C})	Outputs (o_{SLC1})	Outputs (o_R)	Outputs (o_{CNC})
• Cut Part (CNC1 and CNC2) • Load part 1 (R) • Load part 2 (R) • Part 2 Done (SLC1) • Part 2 Done (SLC1)	• Process Part 1 (C1C) • Process Part 2 (C1C)		
Inputs (i_{C1C})	Inputs (i_{SLC1})	Inputs (i_R)	Inputs (i_{CNC})
• Process Part 1 (SLC1) • Process Part 2 (SLC1) • Part Done (CNC1 and CNC2)	• Part 1 Done (C1C) • Part 2 Done (C1C)	• Load Part 1 (C1C) • Load Part 2 (C1C) • UnLoad Part 1 (C1C) • UnLoad Part 2 (C1C)	• Cut Part (C1C)

(a)



(b)

Fig. 9. (a) The inputs, outputs and states of (b) the tested portion of the RFT, where the parentheses denote the other communicating entity. SLC1 is the version of the SLC specifically designed for the testing covered in this paper, and W stands for workpiece.

The CNCs will remain in their *actual* form throughout testing, but the *actual* robot will be replaced by a *simulated* robot. The behaviors of the CNCs and the robot are functions $o_{CNC} = f_{CNC}(i_{CNC}, x_{CNC}, x_{workpiece})$ and $o_R = f_R(i_R, x_R, x_{workpiece})$, respectively, where o, x , and i are given in Fig. 9. The state of the workpiece $x_{workpiece}$ is conveyed through the workpiece's Newtonian effects.

The CNCs and robot exist in the *physical domain* completely at first. Similar to C1C, their *Newtonian effects* are mapped to *procedural effects*. When the CNC is finished cutting and the gripper opens, the "process done" command is then sent to C1C. The open gripper is a *Newtonian effect*, but the command being sent is a *procedural effect*. The CNCs and robot differ from both C1C and SLC1 because they have *spatial essence*. The spatial essences are the actual physical machines, and the effects/outputs are covered in Fig. 9.

It is assumed in the RFT example that the controllers can see the internal states of the CNCs and robots because C1C as originally written [34] tracks their internal states. An explicit communication interface with inputs and outputs could also be used.

B. Implementation of Simulated Region

To apply HPS concepts, the following three testing scenarios were applied to the RFT: the actual Cell 1 robot was replaced with a simulated robot, the actual Cell 1 was replaced entirely



Fig. 10. Visualization of the three-axis overhead gantry that represents the SSC.

with a simulated cell, and a simulated automated supply cell was added to the system. These three case studies (replacing a component, replacing a cell, and adding a new cell) demonstrate some of the capabilities of an HPS.

The simulated supply cell (SSC) and the simulated cell that replaced Cell 1 use behavioral models instead of mathematical or stochastic simulation models. The behavioral model provides the correct signal response given a specific set of signal inputs; it is not a detailed model based on equations of motion. The behavioral models have geometric and kinematic descriptions that exist in a game engine environment. The screen shots in the following sections are examples of what the model's visualizations look like in their functioning environment. The simulated robot, however, is a simulation software package obtained from the vendor (Fanuc Robotics) of the component it represents. For this reason, it is a much higher fidelity simulation with program code that is directly portable to the actual Fanuc robot. Understandably, detailed simulation models give a more accurate prediction of system performance for various scenarios. From the standpoint of a proof of concept, however, individual component simulation fidelity, though important, is not the focus of this paper.

1) **Simulated Cell Addition:** The simulated cell addition will be covered in detail because it includes all of the important principles of an HPS. Adding a simulated cell requires signal emulation and “plug and play” testing. The simulated cell addition also includes birthing an entity which is the most challenging of the transitions (discussed in Definition 13).

Adding the simulated cell represents a scenario in which a process planner may want to automate a loading process that is currently manual. The process planner can then test with simulation different supply cells manufactured by different companies. Testing allows the process planner to make a more informed decision, while also decreasing the total amount of time for integration.

This situation corresponds with scenario 2, where the system starts in an actual form with physical processes in place and follows the approach described in Section III-D.

- 1) **Identify the entity to be added.** The SSC was added to create an automated loading station for the workpiece. (Fig. 10 shows the visualization of the supply cell.)
- 2) **Define the essence and effect.** The SSC is an overhead gantry that exists completely in the *digital domain*, which

TABLE I
THE ESSENCES AND EFFECTS OF THE SIMULATED SUPPLY CELL

Simulated Supply Cell		
Physical Domain	Spatial Essence	<i>None</i>
	Newtonian Effect	<i>None</i>
Digital Domain	Data Essence	Java simulation
	Procedural Effects	Outputs and states listed in Fig. 11

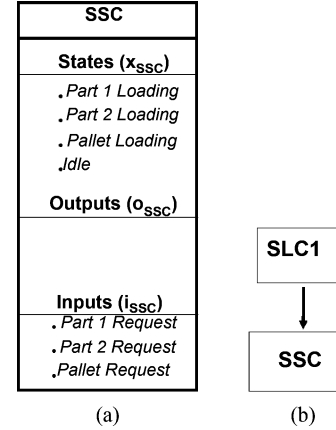


Fig. 11. (a) Inputs, outputs, and states of the SSC and (b) the connection between SLC1 and SSC.

means that it has data essence which is the program itself, and its procedural effects are its changes in state (Table I).

- 3) **Define the behavior of the entity.** The SSC receives requests from SLC1. The SSC starts the process by transporting a workpiece that only exists in the digital domain (Fig. 5, region C) (step 3, scenario 2). The behavior of the SSC is a function $o_{SSC} = f_{SSC}(i_{SSC}, x_{SSC})$, where o, x , and i are given in Fig. 11.
- 4) **Define the interface.** The SSC has an OPC interface.
- 5) **Build the simulation.** The SSC was built using both an open source game engine (Blender3d) and a signal emulator written in Java.
- 6) **Deactivate the actual entity.** Because this is an addition there is no need for deactivation.
- 7) **Connect the simulation to the test setup.** The SSC was connected using the same OPC interface as the existing cells.
- 8) **Modify the system as needed, such that it can function correctly within the larger system and handle the transition from simulated workpiece to actual workpiece.** The workpiece leaves the SSC (Fig. 4, regions C to D), where it is transported by the conveyor to Cell 1 (Fig. 4, region C). The transition out of the simulation into the physical domain means that birthing has occurred. The workpiece never had *spatial essence*, but it must now have *Newtonian effect*. Newtonian effect is accomplished through workpiece emulation.
- 9) **Run the test setup for validation.** The addition of the SSC did require some modifications to be made to SLC1 (Section IV-B, scenario 2). These are the same changes that will be needed when the actual supply cell is added. The integration of the actual supply cell should then be faster



Fig. 12. A screen shot of the Roboguide visualization for the simulated robot.

TABLE II
THE ESSENCES AND EFFECTS OF THE SIMULATED ROBOT

Simulated Robot		
Physical Domain	Spatial Essence	<i>None</i>
	Newtonian Effect	<i>None</i>
Digital Domain	Data Essence	Roboguide simulation
	Procedural Effects	Outputs and states listed in Fig. 9a

because the control logic is already validated to work with the SSC.

2) *Simulated Robot Integration*: Replacing the real robot with a simulation represents scenario 2, where a manufacturing process planner detects a malfunctioning robot. The planner then has the option to purchase a new robot with similar functionality. With the HPS approach in mind, the planner could acquire multiple simulated robots (from different vendors) and compare and contrast them before making a final selection. The steps for the approach are the same as the SSC in Section IV-B1.

The simulated robot has no *spatial essence*; it exists solely in the *digital domain* (Fig. 12 shows the visualization of the simulated robot) and has the *procedural effects* that match the inputs and outputs of its actual counterpart (Table II). The Newtonian effects (gripping the workpiece) are replaced by procedural effects that allow the simulated robot to interact with an emulated workpiece.

During the integration of the simulated robot, no changes were made to the hierarchy or the control logic. The system functionally behaved as if the robot was real. This testing setup means that any scenario run on the system should be a good indicator of how the real system (with emulated workpieces) will behave.

The simulated robot differs from the other simulated components in that the simulation is from the vendor of the actual component (Fanuc Robotics). Fanuc has a simulation software called Roboguide that simulates the motion as well as the control structure and interface. Programs written in Roboguide can be ported directly from the simulation to a real Fanuc robot.

The Roboguide simulation software is proprietary and thus its source code is closed. Proprietary restrictions are typical in most simulation software, in that component vendors do not

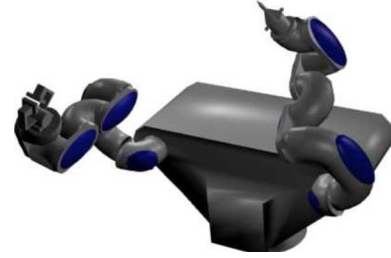


Fig. 13. A screen shot of the simulated Motoman.

TABLE III
THE ESSENCES AND EFFECTS OF THE SIMULATED CELL (MOTOMAN)

Simulated Cell		
Physical Domain	Spatial Essence	<i>None</i>
	Newtonian Effect	<i>None</i>
Digital Domain	Data Essence	Blender and Java simulation
	Procedural Effects	Outputs and states listed in Fig. 14a

want competing vendors to have access to detailed information about their product. Fanuc does, however, supply a PC development kit (PCDK) that makes it possible to communicate with Roboguide. PCDK was used to write an interface between the DeviceNet card and Roboguide, allowing the simulation to be connected over DeviceNet in exactly the same fashion as the actual Fanuc robot.

3) *Simulated Cell Integration*: Replacing the real cell with a simulated cell also represents scenario 2, where a process planner may want to try a very different manufacturing solution. The solution in this case is to replace the two CNCs and one robot with a single robot. The new approach may be more expensive, but it could improve the overall system performance (throughput, quality, etc.). The steps to accomplish the integration are the same as those outlined above for the SSC. Using the simulated cell allows testing of the control logic as well as the communication protocol. Testing will reduce both risk and the time for integration.

The emulated Motoman DA20 was developed to replace Cell 1 (Fig. 13 shows the visualization of the Motoman). The Motoman is a 13 degree-of-freedom two-armed robot that could potentially replace both CNCs and the existing robot by holding the workpiece with one arm and processing it with the other. The Motoman's inputs and outputs are identical to C1C, therefore, SLC1 remains unchanged and unaware of the switch. The Motoman was added to the RFT by disconnecting the Cell 1 controller from SLC1. Disconnection was done by removing the Ethernet cable that enables OPC communication. The Motoman was then connected over Ethernet, using the same OPC outputs and inputs as C1C.

Like the simulated robot, the Motoman exists purely in the *digital domain* (Table III) and does not have Newtonian effect. The behavior of the Motoman is a function $o_{moto} = f_{moto}(i_{moto}, x_{moto})$, where o, x , and i are given in Fig. 14.

Integration of the simulated cell required no change to SLC1 or the existing OPC communication.

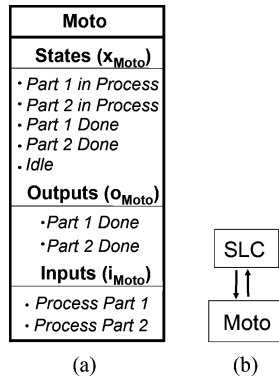


Fig. 14. (a) A schematic of the states, inputs, and outputs of the Motoman (Moto) and (b) how it connects to the existing hierarchy.

C. The Applied Conceptual Architecture

The SSC, simulated robot, and simulated cell are computer emulated entities that maintained data essence and procedural effect (Fig. 4). In the case of the simulated robot and the simulated cell, workpieces entered the simulation through *ghosting*, and after being processed, the workpieces left the simulation through *birthing*. The simulated robot differs from the other two simulations in that it came from the vendor. The simulated robot demonstrates that the modular simulations needed for an HPS can be created without requiring the vendor to open up their proprietary simulation software, provided that an API is available (such as PCDK). If the vendor is not involved, the manufacturing process planner or a third party could create these modularized simulations with low-level interfaces.

V. CONCLUSION AND FUTURE WORK

In this paper, we have put forth an ontology for an HPS, as well as a formalized method for its application. The ontology allows an HPS to be broken down into conceptual components with attributes that are not application specific. The ontology was then applied to three different scenarios on a manufacturing line, including: adding a manufacturing cell that was not present previously, replacing a real robot with a simulated robot, and replacing an existing cell with a simulated cell. The scenarios demonstrate how a workpiece can transition in various different ways, such as starting in a computer simulation and then entering the physical domain, or starting in the physical domain and entering a computer simulation.

The HPS conceptual architecture has all of the benefits of HIL with the added advantage of a formalized approach. HPS has been shown to have specific applicability to a manufacturing process because of its potential for modularity and prelaunch validation. The HPS approach leverages simulation strengths, while lessening the weaknesses that come from a model not being available, or a model being inaccurate. The approach also introduces a feasible solution for combining multiple manufacturing simulations and actual components. This combination of simulated regions and actual regions still allows a workpiece to travel through the system regardless of whether the workpiece is physical or simulated.

A possible next step is to incorporate simulation fidelity metrics into the HPS formalism. Once the accuracy of an HPS is

understood, better decisions can be made about the predictive information that the HPS conveys.

Future work also includes working with component vendors in the development of component simulations capable of functioning within an HPS. There has been an effort in recent years for a basic architecture of the neutral data format AutomationML developed by the companies Daimler, ABB, Siemens, Rockwell, Kuka, Zuhlke, netAllied, and the universities of Magdeburg and Karlsruhe [35]. Such an architecture represents a big step towards creating simulated and real components that could function within an HPS. The authors have also been working closely with OEMs who are interested in using the HPS approach and in working with vendors to enable the solution. Along with the development of simulation software, progress must also be made with encouraging component vendors to develop actual components capable of handling a simulated workpiece so that the entire process can be run without physical workpieces.

REFERENCES

- [1] F. Gu, W. Harrison, D. Tilbury, and C. Yuan, "Hardware-in-the-loop for manufacturing automation control: Current status and identified needs," in *Proc. 3rd Annu. IEEE Conf. Autom. Sci. Eng.*, 2007, pp. 1105–1110.
- [2] M. Bacic, "On hardware-in-the-loop simulation," in *Proc. 44th IEEE Conf. Decision Control*, 2005, pp. 3194–3196.
- [3] M. MacDiarmid and M. Bacic, "Quantifying the accuracy of hardware-in-the-loop simulations," in *Proc. 2007 Amer. Control Conf.*, 2007, pp. 5147–5152.
- [4] P. J. Gawthrop, D. W. Virden, S. A. Neild, and D. J. Wagg, "Emulator-based control for actuator-based hardware-in-the-loop testing," *Control Eng. Practice*, vol. 16, pp. 897–908, 2008.
- [5] C. Boer, A. Verbracke, and H. Veeke, "The possible role of a backbone architecture in real-time control and emulation," in *Proc. 2002 Winter Simulation Conf.*, 2002, pp. 1675–1682, (v2).
- [6] "Structure and behavior of a PABADIS' PROMISE System," White Paper 2008.
- [7] "PABADIS' PROMISE: Work package 8, system simulation, deliverable 8.3 common demonstration system," Tech. Rep. FP6-IST-016649, 2008. [Online]. Available: <http://www.pabadis-promise.org/>
- [8] C. McLean, S. Jain, A. Craens, and D. Kibira, "A virtual manufacturing environment for interoperability testing," in *Proc. Delft Univ. Technol. Conf.*, 2007.
- [9] S. Jain, F. Riddick, A. Craens, and D. Kibira, "Distributed simulation for interoperability testing along the supply chain," in *Proc. 39th Conf. Winter Simulation, WSC'07*, 2007, pp. 1044–1052.
- [10] G. Shao, S. Leong, and C. McLean, "Simulation-based manufacturing interoperability standards and testing," in *Proc. 9th Int. Conf. Progress of Mach. Technol.*, 2009, pp. 283–286.
- [11] D. Kibira and C. R. McLean, "Generic simulation of automotive assembly for interoperability testing," in *Proc. 39th Conf. Winter Simulation, WSC'07*, 2007, pp. 1035–1043.
- [12] C. McLean, F. Riddick, and Y. T. Lee, "An architecture and interfaces for distributed manufacturing simulation," *SIMULATION*, vol. 81, no. 1, pp. 15–32, 2005.
- [13] C. McLean, S. Jain, F. Riddick, and Y. T. Lee, "A simulation architecture for manufacturing interoperability testing," in *Proc. 2007 Summer Comput. Simulation Conf.*, 2007, pp. 601–608.
- [14] L. Pedro, A. Dias, L. Massaro, and G. Caurin, "Dynamic modelling and hardware-in-the-loop simulation applied to a mechatronic project," in *Proc. 19th Int. Congr. Mech. Eng., COBEM'07*, 2007.
- [15] S. Kanai, T. Miyashita, and T. Tada, "A multi-disciplinary distributed simulation environment for mechatronic system design enabling hardware-in-the-loop simulation based on HLA," *Int. J. Interactive Design and Manuf.*, vol. 1, pp. 175–179, 2007.
- [16] H. Hibino and Y. Fukuda, "Emulation in manufacturing engineering processes," in *Proc. 40th Conf. Winter Simulation, WSC '08*, 2008, pp. 1785–1793.

- [17] R. Diogo, C. Vicari, E. Loures, M. Buseti, and E. Santos, "An implementation environment for automated manufacturing systems," in *Proc. 17th World Cong. Int. Fed. Autom. Control*, 2008, p. 10552.
- [18] H. Hibino, T. Inukai, and Y. Fukuda, "Efficient manufacturing system implementation based on combination between real and virtual factory," *Int. J. Prod. Res.*, vol. 44, no. 18, pp. 3897–3915, 2006.
- [19] F. Auinger, M. Vorderwinkler, and G. Buchtela, "Interface driven domain-independent modeling architecture for soft-commissioning and reality in the loop," in *Proc. 1999 Winter Simulation Conf.*, 1999.
- [20] J. Kelsick and T. Vance, "Discrete event simulation implemented in a virtual environment," *J. Mech. Design*, vol. 125, pp. 428–433, 2003.
- [21] *Communication in the Age of Virtual Reality*, F. Biocca and M. Levy, Eds. Mahwah, NJ: Lawrence Erlbaum Associates, 1995.
- [22] M. Lombard and T. Ditton, "At the heart of it all: The concept of presence," *J. Comput.-Mediated Commun.*, vol. 3, no. 2, 1997.
- [23] R. Wells, J. Fisher, Y. Zhou, B. Johnson, and M. Kyte, "Hardware and software considerations for implementing hardware-in-the-loop traffic simulation," in *Proc. 27th Annu. Conf. IEEE Ind. Electron. Soc.*, 2001, pp. 1915–1919.
- [24] D. Ramaswamy, R. McGee, S. Sivashankar, A. Deshpande, J. Allen, K. Rzemien, and W. Stuart, "A case study in hardware-in-the-loop testing: Development of an ECU for a hybrid electric vehicle," *SAE Tech. Paper Series*, vol. 01, 2004.
- [25] W. Grega, "Hardware-in-the-loop simulation and its application in control education," in *Proc. 29th ASEE/IEEE Frontiers in Education Conf.*, 1999, pp. 12b6-7–12b6-12.
- [26] S. Jang, T. Park, and C. Han, "A control of vehicle using steer-by-wire system with hardware-in-the-loop-simulation system," in *Proc. IEEE/ASME Int. Conf. Adv. Intell. Mech.*, 2003.
- [27] W. Lee, M. Yoon, and M. Sunwoo, "A cost- and time-effective hardware-in-the-loop simulation platform for automotive engine control systems," *Proc. Inst. Mech. Eng., Part D: J. Auto. Eng.*, pp. 41–52, 2002.
- [28] S. Heo, K. Park, and H. Ahn, "Design and implementation of hardware-in-the-loop simulator for EHB systems," in *Proc. SICE Annu. Conf.*, 2003.
- [29] T. Park, C. Han, and S. Lee, "Development of the electronic control unit for the rack-actuating steer-by-wire using the hardware-in-the-loop simulation system," *Mechatronics*, vol. 15, pp. 899–918, 2005.
- [30] B. Powell, N. Sureshbabu, and M. D. K. Bailey, "Hardware-in-the-loop vehicle and powertrain analysis and control design issues," in *Proc. Amer. Control Conf.*, 1998, pp. 483–492.
- [31] D. Maclay, "Simulation gets into the loop," *IEE Review*, vol. 43, pp. 109–112, 1997.
- [32] L. Ferrarini and A. Dede, "A mixed-reality approach to test automation function for manufacturing systems," in *Proc. Conf. Innovation, Res., Develop.*, 2009.
- [33] J. Moyne, J. Korsakas, and D. Tilbury, "Reconfigurable factory testbed RFT: A distributed testbed for reconfigurable manufacturing systems," in *Proc. Jpn.-USA Symp. Flexible Autom.*, 2004.
- [34] E. Endsley, E. Almeida, and D. Tilbury, "Modular finite state machines: Development and application to reconfigurable manufacturing cell controller generation," *Control Eng. Practice*, vol. 14, pp. 1127–1142, 2006.
- [35] R. Drath, A. Luder, J. Peschke, and L. Hundt, "Automation ML—The glue for seamless automation engineering," in *Proc. IEEE Int. Conf. Emerging Technol. Factory Autom. ETFA'08*, Sep. 2008, pp. 616–623.



William S. Harrison received the B.S. degree in mechanical engineering from the University of Michigan, Ann Arbor, and the M.S. degree in materials science engineering from the University of Florida, Gainesville. Currently, he is working towards the Ph.D. degree in mechanical engineering at the University of Michigan.

His interests include virtual fusion, virtual reality, simulation integration, and game engine utilization.



Dawn M. Tilbury (M'91–SM'02–F'08) received the B.S. degree in electrical engineering (*summa cum laude*) from the University of Minnesota in 1989, and the M.S. and Ph.D. degrees in electrical engineering and computer sciences from the University of California, Berkeley, in 1992 and 1994, respectively.

In 1995, she joined the faculty of the University of Michigan, Ann Arbor, where she is currently a Professor of Mechanical Engineering with a joint appointment in Electrical Engineering. Her research interests lie broadly in the area of control systems, including applications to robotics and manufacturing systems.

Prof. Tilbury is currently a member of the ASME Dynamic Systems and Control Division Executive Committee and is a life member of SWE.



Chengyin Yuan received the Ph.D. degree in mechanical engineering from the University of Illinois at Urbana–Champaign, Urbana.

He is currently VP at the Beijing Pride Power Systems Technology, Ltd., a startup company to provide advanced power storage solutions for electric vehicle. Prior to joining Pride, he was a Senior Researcher at General Motors (GM) Global Research and Development Center, Warren, MI. At GM, he has worked on various areas, including virtual manufacturing, automation system design and control, sustainable manufacturing, and electric vehicle manufacturing. His work has resulted in more than 30 technical publications and more than ten U.S. and international patent applications. Throughout his career, he has collaborated with several national and international universities as well as companies including the University of Michigan, Ann Arbor, Chalmers University of Technology, Sweden, and Siemens Energy and Automation.

Dr. Chengyin has received numerous awards including the 2010 SME Richard E. Morley OYME Award, GM's "Best of the Best" Chairman's Honor Award, and the Boss Kettering Award. Outside of his career, he has been a For Inspiration and Recognition of Science and Technology (FIRST) Robotics mentor and is an active member of SME.

Dr. Chengyin has received numerous awards including the 2010 SME Richard E. Morley OYME Award, GM's "Best of the Best" Chairman's Honor Award, and the Boss Kettering Award. Outside of his career, he has been a For Inspiration and Recognition of Science and Technology (FIRST) Robotics mentor and is an active member of SME.